# Video 02 — How hardware enables AI: tutorial companion

**Video file:** BlueImpulse_Video02_How_Hardware_Enables_AI.mp4
**Viewing position:** 2 of 5
**Duration:** 10:00 (600 seconds)
**Format:** MP4, 1920 × 1080, 60 fps
**File size:** 48,161,007 bytes (48.16 MB)

**Prerequisites:** Know that software executes instructions and stores data. Chapter 1 — The Fundamentals supplies this foundation; Video 05 is an optional preparatory lesson.

**Learning goals:**

- Trace sensing to action
- Explain memory and processing roles
- Diagnose a hardware bottleneck

**Chapter and subtopic connections:**

- Chapter 3 — Hardware / Hardware gives software a body
- Chapter 3 — Hardware / Memory: capacity versus speed
- Chapter 3 — Hardware / The hardware behind an AI model
- Chapter 3 — Hardware / Overlap work with a pipeline
- Chapter 3 — Hardware / Sense, decide, act, sense again
- Chapter 6 — Strengths & Limitations / Four physical budgets

**Apply in the learning system:** Complete ACT-03 in the Study Guide / Markdown activity file. Use Test Bank questions TF09–TF12, MC09–MC12 and OE03 for review. The six-chapter PPT deck introduces the same chapter concepts through classroom discussion.

**Optional interactive companion:** ROVERLAB lets you apply these ideas by building a robot, connecting its network and comparing stimulus-response rules. Play at https://roverlab-blue-impulse.vercel.app and use the ROVERLAB activity in the Study Guide. It is a simplified rule-and-feedback model, not a neural network or the numerical Q-learning algorithm taught in Video 01.

**Materials:** Paper and a calculator are sufficient. The optional supporting Python demonstration uses Python 3.9 or newer with no third-party packages or account.

```
python3 supporting/BlueImpulse_Demonstrations.py --topic hardware
```

## How to learn with this tutorial

Watch the opening problem, write a prediction, and follow the labelled mechanism. Pause during each learning check before reading the feedback. After the worked example, change one input and explain why the result changes. The tutorial is self-contained: all inputs, instructions and expected outputs are below.

## Chapter timestamps and teaching route

| Time | Teaching step | Chapter / subtopic |
|---|---|---|
| 00:00–00:28 | Pip's plan is waiting | Chapter 3 — Hardware / Hardware gives software a body |
| 00:28–01:04 | Today's route | Chapter 3 — Hardware / Hardware gives software a body |
| 01:04–01:52 | Follow the physical path | Chapter 3 — Hardware / Hardware gives software a body |
| 01:52–02:35 | Use the kitchen analogy | Chapter 3 — Hardware / Memory: capacity versus speed |
| 02:35–03:47 | The processor may be waiting | Chapter 3 — Hardware / Memory: capacity versus speed |
| 03:47–04:39 | Match the processor to the work | Chapter 3 — Hardware / The hardware behind an AI model |
| 04:39–05:45 | Overlap work, increase throughput | Chapter 3 — Hardware / Overlap work with a pipeline |
| 05:45–06:49 | A signal is not motor power | Chapter 3 — Hardware / Sense, decide, act, sense again |
| 06:49–07:35 | Close the loop | Chapter 3 — Hardware / Sense, decide, act, sense again |
| 07:35–08:32 | Will a faster GPU fix this? | Chapter 6 — Strengths & Limitations / Four physical budgets |
| 08:32–09:32 | Hardware enables every step | Chapter 3 — Hardware / The hardware behind an AI model |
| 09:32–10:00 | Pip completes the shift | Chapter 3 — Hardware / Hardware gives software a body |

## Reproduce the pipeline demonstration

Assume four equal one-cycle stages: F = fetch, D = decode, E = execute, W = write-back. Instructions are independent and no stage stalls. Record the occupied stage in each cycle:

| Instruction | Cycle 1 | Cycle 2 | Cycle 3 | Cycle 4 | Cycle 5 | Cycle 6 |
|---|---|---|---|---|---|---|
| I1 | F | D | E | W | — | — |
| I2 | — | F | D | E | W | — |
| I3 | — | — | F | D | E | W |

Completion cycles are 4, 5 and 6. Three instructions finish in six cycles, rather than twelve sequential stage cycles. Each instruction still traverses four stages: pipeline throughput and

individual latency are different.

**Practice:** The processor repeatedly waits for data, and a motor driver cannot supply enough current. Does a faster GPU fix both problems?

**Checked answer:** No. Memory movement and actuator power are distinct bottlenecks. Measure the limiting path, improve the memory transfer or working set as appropriate, and size the driver/power system for the actuator. A processor upgrade alone does not establish either fix.

# Guided explanation

## 00:00 — Pip’s plan is waiting

**Reference:** Chapter 3 — Hardware / Hardware gives software a body

Pip has a plan. The wheels still wait. A sensor stream reaches our campus rover faster than its computer can process it. A clever decision still needs memory, processing, connections, and enough power to become an action.

Have you ever wondered what gives an AI decision a physical body? Chapter three, Hardware, will help us follow the whole path and find the actual bottleneck.

## 01:04 — Follow the physical path

**Reference:** Chapter 3 — Hardware / Hardware gives software a body

The subtopic Hardware gives software a body groups hardware by input, processing, output, and storage. For our robot, the sensor supplies observations. Memory holds inputs, model parameters, and working results. The processor evaluates the program or model, and an output communicates the resulting decision.

If the robot needs to move, an actuator must turn that decision into physical motion. Notice that one device can have multiple roles. A touchscreen both displays output and senses input. The diagram is a chain of responsibilities, not a claim that every box must be a separate physical chip.

## 01:52 — Use the kitchen analogy

**Reference:** Chapter 3 — Hardware / Memory: capacity versus speed

Use a kitchen to remember the memory hierarchy. Registers are like the tiny workspace immediately in your hands. Cache keeps useful ingredients close to the cook. Main memory is the larger active workbench, while persistent storage is more like the pantry.

The analogy is about access and capacity, not a literal map of computer components. A huge pantry does not help if ingredients arrive too slowly. Similarly, a processor can wait for data. Capacity asks how much fits. Memory bandwidth asks how quickly data can move. These are different questions.

## 02:35 — The processor may be waiting

**Reference:** Chapter 3 — Hardware / Memory: capacity versus speed

Model weights, inputs, and intermediate results must fit somewhere. A processor cannot use data that has not arrived. When memory bandwidth is insufficient, a fast processor may spend time waiting instead of doing useful arithmetic. Adding storage capacity alone does not automatically remove a memory-transfer bottleneck.

The table also distinguishes cache, main RAM, and secondary storage. They are not interchangeable names for one resource. Both static RAM and dynamic RAM are volatile, although their circuits and refresh requirements differ. For this tutorial, focus on the system question: is the limiting factor how much fits, or how quickly it arrives?

Imagine the processor as a cook waiting for ingredients. Hiring a faster cook does not shorten the time a missing ingredient spends in transit. Likewise, a faster arithmetic unit can still spend time waiting for memory. Measure that waiting before choosing the upgrade.

## 03:47 — Match the processor to the work

**Reference:** Chapter 3 — Hardware / The hardware behind an AI model

Now compare the processor roles in the subtopic Hardware for AI. A CPU handles general control, branches, and diverse tasks. A GPU can accelerate suitable parallel arithmetic workloads. Specialised accelerators can efficiently perform supported tensor operations, but their benefits depend on the actual workload.

Simple tabular reinforcement learning can run on an ordinary CPU. AI does not always require a GPU. Training changes model parameters. Inference uses an existing model to produce an output. A model can be trained on a large server and deployed on a smaller device if the deployment requirements fit.

## 04:39 — Overlap work, increase throughput

**Reference:** Chapter 3 — Hardware / Overlap work with a pipeline

Use laundry to understand pipelining. One load can dry while a later load is washing. In our simplified processor, the stages are fetch, decode, execute, and write. Instruction one enters fetch in cycle one. In cycle two, it decodes while instruction two enters fetch.

Keep stepping forward. The instructions occupy different stages, so several instructions make progress during the same cycle. After the pipeline fills, this ideal example completes one instruction per cycle, assuming no stalls. It does not make an individual four-stage instruction take just one cycle from start to finish.

Real pipelines can pause for dependencies, uncertain branches, or shared resources. These hazards limit the ideal improvement. Use the included Python table to compare completion cycles rather than relying only on the moving picture.

## 05:45 — A signal is not motor power

**Reference:** Chapter 3 — Hardware / Sense, decide, act, sense again

Trace the environmental control loop: measurement becomes a signal, processing chooses an output, and an actuator changes the environment. An analogue sensor may need an analogue-to-digital converter. Many sensors already provide digital measurements. The processor evaluates a decision, then a driver or power stage supplies what the actuator needs.

Some actuators use a digital interface or pulse-width modulation. They do not all require a digital-to-analogue converter. Pulse-width modulation is timed switching, not the same thing as converting a value into an analogue voltage. A control signal can tell a motor what to do without being able to supply the current that moves it.

Pip's control signal communicates what the motor driver should do. The energy that turns a loaded wheel still comes through the power system.

### 06:49 — Close the loop

**Reference:** Chapter 3 — Hardware / Sense, decide, act, sense again

Imagine the robot detects an obstacle, selects a turn, and sends a command to its motor driver. The driver powers the motor, the wheels move, and the sensor measures the changed situation. That final measurement closes the feedback loop. The command alone does not prove the movement succeeded.

A wheel might slip, a sensor might be noisy, or the actuator might reach a mechanical limit. Monitoring and control differ. Displaying a measurement monitors; changing the environment adds control. This loop can use a fixed rule or a learned policy. The hardware path alone does not tell you which.

### 07:35 — Will a faster GPU fix this?

**Reference:** Chapter 6 — Strengths & Limitations / Four physical budgets

Pause and diagnose a system. Its processor repeatedly waits for data, and its motor driver cannot supply enough current. Would replacing the processor with a faster GPU solve both problems? Think before the answer appears. No. Memory movement and actuator power are separate limits that a new GPU does not automatically remove.

Measure the path, identify the limiting stage, and match the intervention to that evidence. Chapter six, Strengths and Limitations, adds power, heat, memory, and physical constraints. An advertised arithmetic rate describes one capability. It is not a complete measurement of the robot's useful performance.

**Check your answer:** Fix the memory path and power stage.

### 08:32 — Hardware enables every step

**Reference:** Chapter 3 — Hardware / The hardware behind an AI model

Trace Pip's whole path: observations enter, memory holds information, processing evaluates it, and outputs communicate or act. Memory capacity and bandwidth both matter. Pipeline throughput is different from the latency of one instruction. Conversion, control signals, actuator power, and processor strengths all affect what the system can do.

Choose hardware by the workload, deadlines, power budget, and physical task. To check your understanding, draw a robot's path from camera to wheel, and label one possible bottleneck at each stage. Then explain why a more powerful chip does not, by itself, create a learning algorithm.

An upgrade should target the constrained part of the task. Check the input rate, memory traffic, compute load, and actuator response as a connected system.

## Finish by explaining, not copying

Complete ACT-03 and the named Test Bank questions, then explain the mechanism without following the highlight. State one assumption of the worked model and one condition that could make the outcome differ in a real system. A correct calculation supports only the conclusion it actually measures.